

Abstract

This short report is a collection of notes used to develop a framework for hyper reduction of viscoplastic finite element models.

Block 4: Introduction to continuum mechanics

Billy Guy

25-12-22

Contents

1	Tensor Fundamentals	2
1.1	Introduction	2
1.2	Matrices, Vectors and Tensors	3
1.3	Symbols and Tensor Operation	4
1.4	Problem set	5
2	Deformation gradient and motion	9
2.1	Introduction	9
2.2	Deformation Gradient	10
2.3	Motion	13

Chapter 1

Tensor Fundamentals

Section References

The references used in this chapter are:

- Continuum Mechanics website [2]
 - Vectors
 - Matrices & Tensors
 - Tensor Notation (Basics)
 - Tensor Notation (Advanced)

This chapter will cover the vector and tensor essentials to understand tensor algebra applied to solid mechanics. Vectors and matrix are essentially containers for scalars. Scalars are ordered in a certain fashion and some operation can be done with them. Tensors are an extension to vector and matrix. In this report, this notation will be used:

- Scalars will be lower case letters, i.e.: $a, b, c, ..$
- Vectors will be lowercase bold letters, i.e.: $\mathbf{x}, \mathbf{v}, \mathbf{a}$ or $\mathbf{x}, \dot{\mathbf{x}}, \ddot{\mathbf{x}}$
- Matrix will be uppercase bold letters: $\mathbf{A}, \mathbf{C}, \mathbf{X}$

1.1 Introduction

This chapter is a basis for tensor definition and operation. It is not an in depth review of linear or tensor algebra but rather a simple, short take on the required notions for continuum mechanics.

1.2 Matrices, Vectors and Tensors

A vector is defined as:

$$\mathbf{v} = \begin{pmatrix} i \\ \dots \\ n \end{pmatrix}$$

A matrix is defined as:

$$\mathbf{M} = \begin{bmatrix} 1 & 2 & \dots & n \\ 2 & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ m & \dots & \dots & mn \end{bmatrix}$$

The dot product of two vector is:

$$\mathbf{a} \cdot \mathbf{b} = a_i b_i = a_1 b_1 + a_2 b_2 + a_3 b_3 = c$$

Above, the example use tensor notation and c is a scalar. That means that the use of subscript defines the size of each container (vector = 1st order tensor) and how they interact with each other.

Key Concept 1.2.1: Einstein notation

There is one relatively simple rule with indices: if an index is repeated twice, it is summed. For example:

$$c_{ij} = a_i b_j \neq a_1 b_1 + a_2 b_2 + a_3 b_3$$

instead, $a_i b_j$ yields a matrix or a second order tensor:

$$c_{ij} = a_i b_j = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

As seen in the concept above, the way the indices are called defines how they are used. This introduces also the concept of tensor product and contraction. The first example $a_i b_i$ can be seen as the tensor contraction of two 1st order tensor:

$$\mathbf{a} : \mathbf{b} = a_i b_i = a_1 b_1 + a_2 b_2 + a_3 b_3$$

The later can be seen as the tensor product or dyadic product of two 1st order tensor:

$$\mathbf{a} \otimes \mathbf{b} = \begin{bmatrix} a_1 b_1 & a_1 b_2 & a_1 b_3 \\ a_2 b_1 & a_2 b_2 & a_2 b_3 \\ a_3 b_1 & a_3 b_2 & a_3 b_3 \end{bmatrix}$$

For the sake of comprehension, the same operations can be done with vector-matrix notation. Consiering both vectors $\mathbf{a}$ and $\mathbf{b}$ are column vectors :

$$\mathbf{a}^T \mathbf{b} = \mathbf{a} \cdot \mathbf{b} = \mathbf{a} : \mathbf{b}$$

and

$$\mathbf{a} \mathbf{b}^T = \mathbf{a} \otimes \mathbf{b}$$

Here, the difference between the tensor contraction and dot product is that tensor contraction works on higher tensor dimensions while the dot product only work on a combination of vector. The same goes for the dyadic product. An example of higher order tensor contraction would be the combination of the elastic stiffness tensor (4th order) and strain tensor (2nd order) that yields the stress tensor (2nd order):

$$\boldsymbol{\Sigma} = \mathbb{C} : \boldsymbol{E}$$

Important note: here, this expression could be reduced to a simpler expression using the symetries in $\mathbb{C}$ and using the difinition of stress and strain in Voigt notation, yielding:

$$\boldsymbol{\sigma} = \boldsymbol{C} : \boldsymbol{\varepsilon}$$

Voigt notation will be in [chapter n](#).

1.3 Symbols and Tensor Operation

Another use of index notation and helpful for expression contraction is the Kronecker Delta (δ_{ij}). The Kronecker Delta takes two inputs (i and j) and returns one if both are greater than zero (should always be) and are equal:

$$\delta_{ij} = \begin{cases} 1 & i=j, \\ 0 & \text{else} \end{cases}$$

Example 1.3.1: Identity matrix and the Kronecker Delta

The identity matrix in tensor notation is the Kronecker Delta.

$$\boldsymbol{I} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \delta_{ij}$$

Important note: the identity matrix is not the Kronecker Delta since it is a scalar (either 0 or 1).

Another symbol called the permutation symbol (or the Levi-Civita symbol) is a scalar defined as $\varepsilon_{ijk}\dots$. The $\dots$ means that it exists for higher indices the symbol and works with the same logic. The idea is that if it follows a certain ordering the return value is either 1, -1 or 0 :

$$\epsilon_{ijk} = \begin{cases} +1 & \text{if } (i, j, k) \text{ } (1, 2, 3), (2, 3, 1) \text{ or } (3, 1, 2) \rightarrow \text{even permutation ,} \\ -1 & \text{if } (i, j, k) \text{ } (3, 2, 1), (2, 1, 3) \text{ or } (1, 3, 2) \rightarrow \text{odd permutation ,} \\ 0 & \text{if any two indices are equal} \end{cases}$$

The Levi-Civita symbol can be used to define the cross product of two vector for example, let $\mathbf{a} \times \mathbf{b}$, thus:

$$c_i = \epsilon_{ijk} a_j b_k$$

Example 1.3.2: Cross product using Levi-Civita symbol ϵ_{ijk}

To find each component of the vector $\mathbf{c} = \mathbf{a} \times \mathbf{b}$, we take each component that are not equal to 0 in the permutation, thus:

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} \epsilon_{1jk} a_j b_k \\ \epsilon_{2jk} a_j b_k \\ \epsilon_{3jk} a_j b_k \end{bmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

1.4 Problem set

Problem 1.1 *Einstein summation*

Given the following vectors:

$$\mathbf{a} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$$

Find $a_i b_i$.

Answer: Since index i is repeated, we sum all components on i . Thus, it becomes a dot product. The result is:

```
a = np.array([2,-1,3])
b = np.array([1,4,-2])
c = np.dot(a,b)
```

$$\begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 4 \\ -2 \end{bmatrix} = -8$$

Problem 1.2 Matrix-Vector product

Given matrix A and vector x , compute $y_i = A_{ij}x_j$ for each component

Answer: Since index j is repeated, each y_i component will be a sum of combination of $A_{ij}b_j$.

```
from sympy import MatrixSymbol
A = MatrixSymbol('A', 3, 3)
x = MatrixSymbol('x', 3, 1)
y = MatrixSymbol('y', 3, 1)
y = A * x
```

$$A = \begin{bmatrix} A_{0,0} & A_{0,1} & A_{0,2} \\ A_{1,0} & A_{1,1} & A_{1,2} \\ A_{2,0} & A_{2,1} & A_{2,2} \end{bmatrix} \quad x = \begin{bmatrix} x_{0,0} \\ x_{1,0} \\ x_{2,0} \end{bmatrix}$$
$$y = Ax$$
$$y = \begin{bmatrix} A_{0,0}x_{0,0} + A_{0,1}x_{1,0} + A_{0,2}x_{2,0} \\ A_{1,0}x_{0,0} + A_{1,1}x_{1,0} + A_{1,2}x_{2,0} \\ A_{2,0}x_{0,0} + A_{2,1}x_{1,0} + A_{2,2}x_{2,0} \end{bmatrix}$$

As expected, i is fixed on A and summed on j for both A and x . Note that Sympy indicate a double index notation for x even though that it is a vector. However, the second index doesn't change.

Problem 1.3 Kronecker Delta

Prove that $\delta_{ij}\delta_{jk} = \delta_{ik}$

Answer: using a range of $i, j, k \in (1, 2, 3)$ δ_{ij} or δ_{jk} can only be non null if $i = j$ or $j = k$. Then only combination of $i = k$ will yield a result because by definition $i = j = k$ in this case. For example:

```
# Create a Kronecker Delta function
def deltakron(i,j):
    if i==j:
        return 1
    else:
        return 0

# Intialize sum variables
sum = 0
sum2 = 0

# Compute using i-j-k
for i in range(3):
    for j in range(3):
```



```

        for k in range(3):
            sum += deltakron(i,j)*deltakron(j,k)

# Compute using i-k
for i in range(3):
    for k in range(3):
        sum2 += deltakron(i,k)

```

As expected, $sum = 3$ and $sum2 = 3$

Problem 1.4 *Cross product using Levi-Civita symbol ϵ_{ijk}*

Compute the cross product of two vectors $\mathbf{a} \times \mathbf{b}$ using the Levi-Civita symbol.

Answer: In reference to the worked example, see section 1.3. The numeric resolution gives:

```

from sympy import Eijk

# Initialize two 3x1 vectors
a = MatrixSymbol('a', 3, 1)
b = MatrixSymbol('b', 3, 1)

# Since MatrixSymbol doesnt support insertion, a list and a sum are initialized
c = []
# Loop over indices and sum
for i in range(3):
    sum = 0
    for j in range(3):
        for k in range(3):
            sum += Eijk(i,j,k)*a[j]*b[k]

    c.append(sum)

```

$$c = \begin{bmatrix} a_{1,0}b_{2,0} - a_{2,0}b_{1,0} \\ -a_{0,0}b_{2,0} + a_{2,0}b_{0,0} \\ a_{0,0}b_{1,0} - a_{1,0}b_{0,0} \end{bmatrix}$$

Problem 1.5 *Trace*

Compute the trace of A using tensor notation

Answer: The trace is computed as A_{ii} . Since i is repeated twice it is summed. Thus:

```

# Intialize a 3x3 matrix
A = MatrixSymbol('A',3,3)

```

```
# Initialize a sum to compute trace
sum = 0

for i in range(3):
    sum += A[i,i]

# Compute trace using numpy
sum2 = np.trace(A)
```

As expected, $sum = A[0, 0] + A[1, 1] + A[2, 2]$ and $sum2 = A[0, 0] + A[1, 1] + A[2, 2]$

Chapter 2

Deformation gradient and motion

Section References

The references used in this chapter are:

- Belytschko [1]
 - 3.2.2-3.2.3
 - 3.2.6
 - 3.7.1
- Continuum Mechanics website [2]
 - Deformation Gradient
 - Polar Decomposition

2.1 Introduction

Continuum mechanics introduce the concepts of **material** and **spatial** definition. See fig. 2.1. The difference between material and spatial is the definition of what is observed:

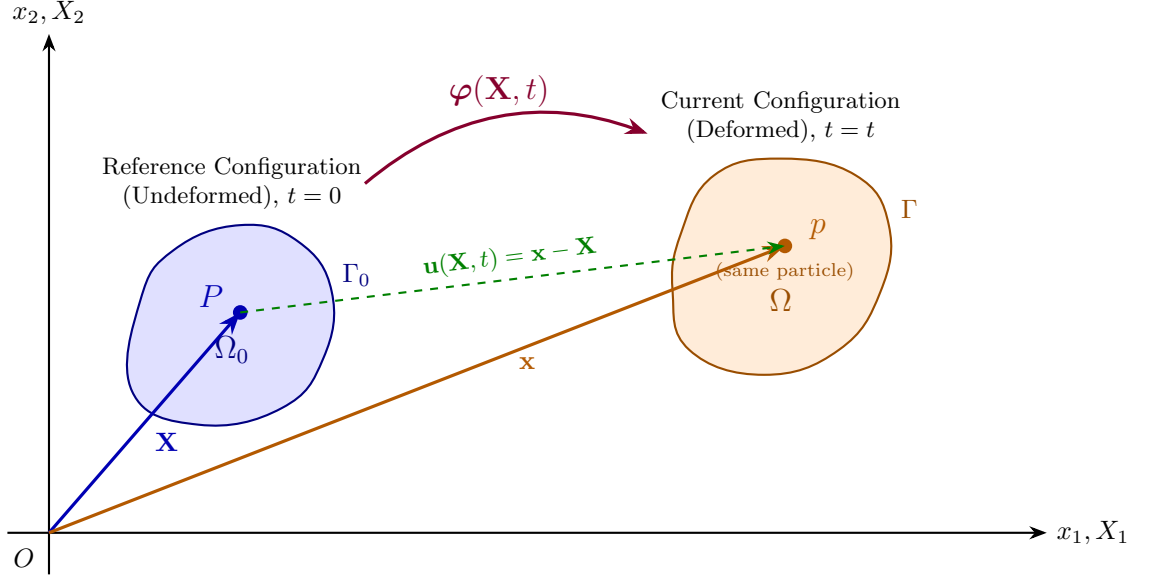


Figure 2.1: Material and spatial definition of continuum

Key Concept 2.1.1: Material vs Spatial description

- Material $\rightarrow$ Lagrangian: Following a person in a crowd. Typical in solid mechanics (FEA).
- Spatial $\rightarrow$ Eulerian: Fixing a point in a crowd and seeing who passes through. Typical in fluid mechanics (CFD).

The configuration of a body is defined by the symbol Ω and its boundaries are represented by Γ . The subscript $_0$ indicates an initial state. To go from Ω_0 to Ω , the body goes from $\mathbf{X}$ to $\mathbf{x}$. Essentially $\mathbf{X}$ means which particle (where is it initially) and $\mathbf{x}$ means where is it now? The idea is the same for the definition of any point $\mathbf{P}$ and $\mathbf{p}$. Then, how $\mathbf{x}$ relates to $\mathbf{X}$ is defined by the deformation map $\varphi(\mathbf{X}, t)$ and its displacement $\mathbf{u} = \mathbf{x} - \mathbf{X}$. The deformation map can be seen as a tracker in the "crowd" example above: Given this initial point in space, what happened to whatever went through this point initially.

2.2 Deformation Gradient

The deformation gradient $\mathbf{F}$ is simply the derivative of each component of the deformed vector from the deformed configuration: $\mathbf{x}$ with respect to each com-

ponent of the reference configuration $\mathbf{X}$:

$$F_{ij} = x_{i,j} = \frac{\partial x_i}{\partial X_j} = \begin{bmatrix} \frac{\partial x_1}{\partial X_1} & \frac{\partial x_1}{\partial X_2} & \frac{\partial x_1}{\partial X_3} \\ \frac{\partial x_2}{\partial X_1} & \frac{\partial x_2}{\partial X_2} & \frac{\partial x_2}{\partial X_3} \\ \frac{\partial x_3}{\partial X_1} & \frac{\partial x_3}{\partial X_2} & \frac{\partial x_3}{\partial X_3} \end{bmatrix} \quad (2.1)$$

Example 2.2.1: Rigid body motion

The displacement function is :

$$\mathbf{u} = \begin{bmatrix} x = X + 5 \\ y = Y + 2 \end{bmatrix}$$

Here, $\mathbf{F}$ is simply $\mathbf{I}$ because

$$\mathbf{F} = \frac{x_i}{X_j} = \begin{bmatrix} \frac{\partial(X+5)}{\partial(Y+2)} & \frac{\partial(X+5)}{\partial Y} \\ \frac{\partial(Y+2)}{\partial X} & \frac{\partial(Y+2)}{\partial Y} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Thus, the body is not deformed hence the title *Rigid body*.

Example 2.2.2: Rotation

The displacement function is:

$$\mathbf{u} = \begin{bmatrix} x = X \cos \theta - Y \sin \theta \\ y = X \sin \theta + Y \cos \theta \end{bmatrix}$$

$\mathbf{F}$ is then:

$$\mathbf{F} = \frac{\partial x_i}{\partial X_j} = \begin{bmatrix} \frac{\partial(X \cos \theta - Y \sin \theta)}{\partial(X \sin \theta + Y \cos \theta)} & \frac{\partial(X \cos \theta - Y \sin \theta)}{\partial Y} \\ \frac{\partial(X \sin \theta + Y \cos \theta)}{\partial X} & \frac{\partial(X \sin \theta + Y \cos \theta)}{\partial Y} \end{bmatrix}$$

$$\mathbf{F} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Key Concept 2.2.1: Deformation gradient and hourglassing

Sometimes, displaced bodies that undergo true deformation do not yield deformation when computed. This is the case with hourglassing, a numerical error in finite element when elements do not yield any deformation when in fact they deform. It is typical in dominant bending problems and when selected elements use reduced integration. While this is out of topic, it is an important concept and gives a good example of the calculation of deformation gradient. Example adapted and taken from [2].

The displacement function is given by:

$$\mathbf{u} = \begin{bmatrix} x = X + \frac{1}{4}XY + 5 \\ Y + 4 \end{bmatrix}$$

The deformation gradient is then:

$$\mathbf{F} = \begin{bmatrix} 1 + \frac{1}{4}Y & \frac{1}{4}X \\ 0 & 1 \end{bmatrix}$$

at $(0,0)$, the initial position of the centre of the square, the deformation gradient is $\mathbf{I}$ since $X = 0$ and $Y = 0$, even though visually the body deforms:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

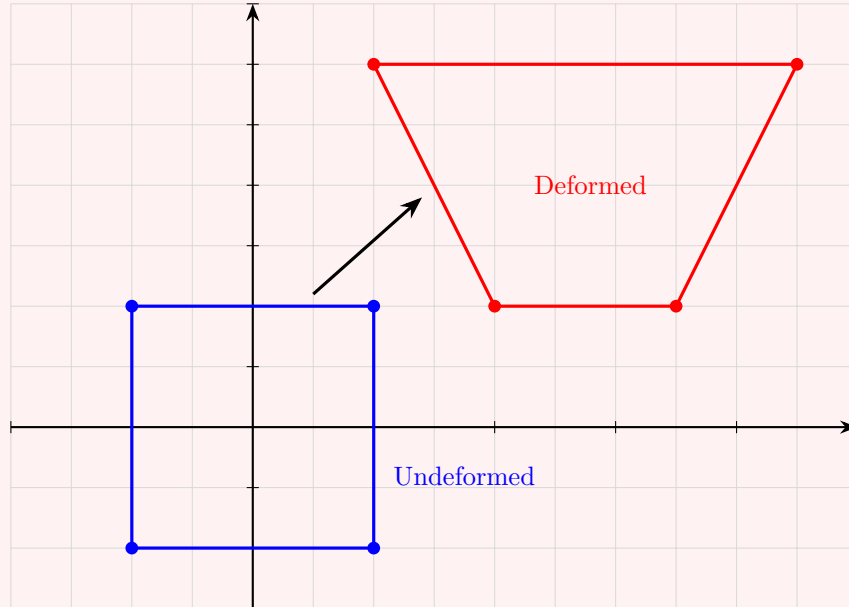


Figure: Example of hourglassing and deformed configuration of a body

2.3 Motion

As described earlier, the displacement of a body is defined by:

$$\mathbf{u} = \mathbf{x} - \mathbf{X} \quad (2.2)$$

and the map from $\mathbf{X}$ to $\mathbf{x}$ is:

$$\phi(\mathbf{X}, t) \quad (2.3)$$

The velocity $\mathbf{v}(\mathbf{X}, t)$ is then, from eq. (2.2) and eq. (2.3):

$$\mathbf{v}(\mathbf{X}, t) = \frac{\partial \phi(\mathbf{X}, t)}{\partial t} = \frac{\partial \mathbf{u}(\mathbf{X}, t)}{\partial t} \equiv \dot{\mathbf{u}} \quad (2.4)$$

in the same fashion, the acceleration is:

$$\mathbf{a}(\mathbf{X}, t) = \frac{\partial^2 \phi(\mathbf{X}, t)}{\partial t^2} = \frac{\partial^2 \mathbf{u}(\mathbf{X}, t)}{\partial t^2} = \frac{\partial \mathbf{v}(\mathbf{X}, t)}{\partial t} \equiv \ddot{\mathbf{u}} \quad (2.5)$$

Bibliography

- [1] Ted Belytschko, W. K. Liu, B. Moran, and Khalil I. Elkhodary. *Nonlinear finite elements for continua and structures*. Hoboken, New Jersey : Wiley, 2014.
- [2] Bob McGinty. Continuum mechanics.